

对于两个字符串,从第一个字符开始比较,当某一个位置的字符不同时,该位置字符较小的串,字典序较小(例如,abc 比 bcd 小);如果其中一个字符串已经没有更多字符,但另一个字符串还没结束,则较短的字符串的字典序较小(例如,hi 比 history 小)。字典序的概念可以推广到任意序列,例如,序列 1,2,4,7 比 1,2,5 小。

学会了字典序的概念之后,本题就不难解决了:就像“求 n 个元素中的最小值”一样,用变量 ans 表示目前为止,字典序最小串在输入串中的起始位置,然后不断更新 ans。

```
#include<stdio.h>
#include<string.h>
#define maxn 105

//环状串 s 的表示法 p 是否比表示法 q 的字典序小
```

```
int less(const char* s, int p, int q) {
    int n = strlen(s);
    for(int i = 0; i < n; i++)
        if(s[(p+i)%n] != s[(q+i)%n])
            return s[(p+i)%n] < s[(q+i)%n];
    return 0; //相等
}
```

```
int main() {
    int T;
    char s[maxn];
    scanf("%d", &T);
    while(T--) {
        scanf("%s", s);
        int ans = 0;
        int n = strlen(s);
        for(int i = 1; i < n; i++)
            if(less(s, i, ans)) ans = i;
        for(int i = 0; i < n; i++)
            putchar(s[(i+ans)%n]);
        putchar('\n');
    }
    return 0;
}
```

3.4 注解与习题

到目前为止,C 语言的核心内容已经全部讲完。理论上,运用前 3 章的知识足以编写大部分算法竞赛程序了。

3.4.1 进位制与整数表示

用 ASCII 编码表示字符。下面来探索一下字符在 C 语言中的表示。从正文中可知，有些特殊的字符需要转义才能表达，例如“\n”表示换行，“\”表示反斜杠，“\"表示引号，“\0”表示空字符，那还有哪些转义符呢？如果在网上搜索一下，或者翻阅任何一本 C 语言参考书，就会发现转义字符表中有如下说法。

提示 3-21：字符还可以直接用 ASCII 码表示。如果用八进制，应该写成“\o”，“\oo”或“\ooo”（o 为一个八进制数字）；如果用十六进制，应该写成“\xh”（h 为十六进制数字串）。

什么是八进制和十六进制呢？我们平时使用的是“逢十进一”的进位制系统，称为十进制（Decimal System）。而在计算机内部，所有事物都是用“逢二进一”的二进制（Binary System）来表示。从表 3-1 很容易看出二者之间的关系。

表 3-1 十进制和二进制的转换关系

十进制	0	1	2	3	4	5	6	7
二进制	0	1	10	11	100	101	110	111

类似地，可以定义八进制和十六进制（注意，在十六进制中，用字符 A~F 表示十进制中的 10~15）。如果操作系统是 Windows，打开“计算器”后，先切换到“科学型”，然后输入一个整数，例如 123，再单击“二进制”按钮，就可以看到其二进制值 1111011、八进制值 173 和十六进制值 7B^①。而语句“printf(“%d %o %x\n”, a)”将把整数 a 分别按照十进制、八进制和十六进制输出。

进制转换与移位运算符。如何把二进制转换为十进制？类似于 $123 = ((1 \times 10) + 2) \times 10 + 3$ ，二进制转换为十进制也可以这样一次添加一位，每次乘以 2： $101_2 = ((1 \times 2 + 0) \times 2 + 1) = 5$ 。在 C 语言中，“乘以 2”也可以写成“<<1”，意思是“左移一位”。类似地，左移 4 位就是乘以 2^4 。

在二进制中，8 位最大整数就是 8 个 1，即 $2^8 - 1$ ，用 C 语言写出来就是 $(1 << 8) - 1$ 。注意括号是必需的，因为“<<”运算符的优先级没有减法高。

补码表示法。计算机中的二进制是没有符号的。尽管 123 的二进制值是 1111011，-123 在计算机内并不表示为 -1111011——这个“负号”也需要用二进制位来表示。

“正号和符号”只有两种情况，因此用一个二进制位就可以了。容易想到一个表示“带符号 32 位整数”的方法：用最高位表示符号（0：正数；1：负数），剩下 31 位表示数的绝对值。可惜，这并不是机器内部真正的实现方法。在笔者的机器上，语句“printf(“%u\n”, -1)”的输出是 4294967295^②。把 -1 换成 -2、-3、-4……后，很容易总结出一个规律：-n 的内部表示是 $2^{32} - n$ 。这就是著名的“补码表示法”（Complement Representation）。

提示 3-22：在多数计算机内部，整数采用的是补码表示法。

为什么计算机要用这样一个奇怪的表示方法呢？前面提到的“符号位+绝对值”的方法

^① 遗憾的是，Linux 下的 GUI 计算器 xcalc 无法进行进制转换。不过很多系统预装了 bc 程序，可以使用“echo 'obase=2; ibase=10; 123 | bc’”把十进制 123 转换成二进制。

^② 请记住这个整数，它等于 $2^{32} - 1$ 。

哪里不好了？答案是：运算不方便。试想，要计算 $1 + (-1)$ 的值（为了简单起见，假设两个数都是带符号 8 位整数）。如果用“符号位+绝对值”法，将要计算 $00000001 + 10000001$ ，而答案应该是 00000000 。似乎想不到什么简单的方法进行这个“加法”。但如果采用补码表示，计算的是 $00000001 + 11111111$ ，只需要直接相加，并丢掉最高位的进位即可。“符号位+绝对值”还有一个好玩的 bug：存在两种不同的 0：一个是 00000000 （正 0），一个是 10000000 （负 0）。这个问题在补码表示法中不会出现（想一想，为什么）。

学到这里，你能解释“int 类型的最小、最大值”了吗？提示：在通常情况下，int 是 32 位的。

3.4.2 思考题

题目 1（必要的存储量）：数组可以用来保存很多数据，但在一些情况下，并不需要把数据保存下来。下面哪些题目可以不借助数组，哪些必须借助数组？请编程实现。假设输入只能读一遍。

- ☐ 输入一些数，统计个数。
- ☐ 输入一些数，求最大值、最小值和平均数。
- ☐ 输入一些数，哪两个数最接近。
- ☐ 输入一些数，求第二大的值。
- ☐ 输入一些数，求它们的方差。
- ☐ 输入一些数，统计不超过平均数的个数。

题目 2（统计字符 1 的个数）：下面的程序意图在于统计字符串中字符 1 的个数，可惜有瑕疵：

```
#include<stdio.h>
#define maxn 10000000 + 10
int main() {
    char s[maxn];
    scanf("%s", s);
    int tot = 0;
    for(int i = 0; i < strlen(s); i++)
        if(s[i] == 1) tot++;
    printf("%d\n", tot);
}
```

该程序至少有 3 个问题，其中一个导致程序无法运行，另一个导致结果不正确，还有一个导致效率低下。你能找到它们并改正吗？

3.4.3 黑盒测试和在线评测系统

黑盒测试。算法竞赛一般采取黑盒测试：事先准备好一些测试用例，然后用它们测试选手程序，根据运行结果评分。除了找不到程序（如程序名没有按照比赛规定取，或是放

错位置）、编译错等连程序都没能运行的错误之外，一些典型的错误类型如下：

- ❑ 答案错（Wrong Answer, WA）。
- ❑ 输出格式错（Presentation Error, PE）。
- ❑ 超时（Time Limit Exceeded, TLE）。
- ❑ 运行错（Runtime Error, RE）。

在一些比较严格的比赛中，输出格式错被看成是答案错，而在另外一些比赛中，则会把二者区分开。在运行时，除了程序自身异常退出（例如，除 0、栈溢出、非法访问内存、断言为假、main 函数返回非 0 值）外，还可能是因为超过了评测系统的资源约束（如内存限制、最大输出限制）而被强制中止执行。有的评测系统会把这些情况和一般的运行错误区分开，但在多数情况下会统一归到“运行错”中。

需要注意的是，超时不一定是因为程序效率太低，也可能是其他原因造成的。例如，比赛规定程序应从文件读入数据，但所写程序却正在等待键盘输入。其他原因包括：特殊数据导致程序进入死循环、程序实际上已经崩溃却没异常退出等。

如果上述错误都没有，那么恭喜你，你的程序通过了测试。在 ACM/ICPC 中，这意味着你的程序被裁判接受（accepted, AC），而在分测试点的比赛中，这意味着你拿到了该测试点的分数。

需要注意的是，一些比赛的测试点可以给出“部分分”——如答案正确但不够优，或者题目中有两个任务，选手只成功完成了一个任务等。不管怎样，得分的前提是不超时、没有运行错。只有这样，程序输出才会参与评分。

在线评测系统（Online Judge, OJ）为平时练习和网上竞赛提供了一个很好的平台。事实上，本书中的练习大都通过 OJ 给出。

首先，要向读者介绍的是历史最悠久、最著名的 OJ：西班牙 Valladolid 大学的 UVaOJ，网址为 <http://uva.onlinejudge.org/>^①。除了收录了早期的 ACM/ICPC 区域比赛题目之外，这里还经常邀请世界顶尖的命题者共同组织网上竞赛，吸引了大量来自世界各地的高手同场竞技。

目前，UVaOJ 网站的题库已经包含了一个特殊的分卷（Volume）——AOAPC II，把本书的配套习题按照易于查找和提交的方式集中在一起，并将逐步提供题目的中文翻译和算法提示。根据读者的反馈，网上题库可能在本书的基础上增加一些有价值的题目，并移除一些不太合适的题目，因此建议读者在做题时直接参考 UVaOJ 的 AOAPC 分卷。3.4.2 节的题目中已经给出了 UVa 题目编号。例如，UVa272 就代表 UVa OJ 中编号为 272 的题目。

其他著名的 OJ 包括国内的 ZOJ（浙江大学），POJ（北京大学），HDOJ（电子科技大学）、俄罗斯的 SGU、Timus、波兰的 SPOJ 等。

3.4.4 例题一览与习题

本章的 5 道例题全部是竞赛题目，在 UVa 上可以提交，如表 3-2 所示。

^① 目前的 UVaOJ 网站与 IE 浏览器兼容性不好，推荐使用 Firefox 浏览器。

表 3-2 例题一览

类 别	题 号	题目名称 (英文)	备 注
例题 3-1	UVa272	Tex Quotes	输入输出函数详解
例题 3-2	UVa10082	WERTYU	常量数组的妙用
例题 3-3	UVa401	Palindromes	字符函数; 常量数组
例题 3-4	UVa340	Master-Mind Hints	用数组统计
例题 3-5	UVa1583	Digit Generator	预处理、查表
例题 3-6	UVa1584	Circular Sequence	字典序

从本章开始, 习题全部通过 UVaOJ 给出。由于样例输入输出很占篇幅, 这里通过文字的方式给出例子, 详细的样例输入输出请读者参考原题。在下面的习题中, 前一半的题目几乎只需要“按照题目说的做”, 但后面的题目需要一些思考甚至灵感。

为了保证学习效果, 请至少独立完成 8 道习题。需要特别注意的是, 由于本书前 4 章中的 C 语言程序需要用 C99 编译器, 而 UVa 中的“ANSI C”是指 C89 编译器, 请在提交时选择 C++ 语言。本书前 4 章中介绍的 C 语言全部和 C++ 兼容, 所以源码可以不加修改地用 C++ 编译器编译通过。

习题 3-1 得分 (Score, ACM/ICPC Seoul 2005, UVa1585)

给出一个由 O 和 X 组成的串 (长度为 1~80), 统计得分。每个 O 的得分为目前连续出现的 O 的个数, X 的得分为 0。例如, OOXOXXOOO 的得分为 1+2+0+0+1+0+0+1+2+3。

习题 3-2 分子量 (Molar Mass, ACM/ICPC Seoul 2007, UVa1586)

给出一种物质的分子式 (不带括号), 求分子量。本题中的分子式只包含 4 种原子, 分别为 C, H, O, N, 原子量分别为 12.01, 1.008, 16.00, 14.01 (单位: g/mol)。例如, C₆H₅OH 的分子量为 94.108g/mol。

习题 3-3 数数字 (Digit Counting, ACM/ICPC Danang 2007, UVa1225)

把前 n ($n \leq 10000$) 个整数顺次写在一起: 123456789101112... 数一数 0~9 各出现多少次 (输出 10 个整数, 分别是 0, 1, ..., 9 出现的次数)。

习题 3-4 周期串 (Periodic Strings, UVa455)

如果一个字符串可以由某个长度为 k 的字符串重复多次得到, 则称该串以 k 为周期。例如, abcabcabcabc 以 3 为周期 (注意, 它也以 6 和 12 为周期)。

输入一个长度不超过 80 的字符串, 输出其最小周期。

习题 3-5 谜题 (Puzzle, ACM/ICPC World Finals 1993, UVa227)

有一个 5*5 的网格, 其中恰好有一个格子是空的, 其他格子各有一个字母。一共有 4 种指令: A, B, L, R, 分别表示把空格上、下、左、右的相邻字母移到空格中。输入初始网格和指令序列 (以数字 0 结束), 输出指令执行完毕后的网格。如果有非法指令, 应输出 “This puzzle has no final configuration.”, 例如, 图 3-5 中执行 ARRBBL0 后, 效果如图 3-6 所示。

T	R	G	S	J
X	D	O	K	I
M		V	L	N
W	P	A	B	E
U	Q	H	C	F

图 3-5 执行 ARRBBL0 前

T	R	G	S	J
X	O	K	L	I
M	D	V	B	N
W	P		A	E
U	Q	H	C	F

图 3-6 执行 ARRBBL0 后

习题 3-6 纵横字谜的答案 (Crossword Answers, ACM/ICPC World Finals 1994, UVa232)

输入一个 r 行 c 列 ($1 \leq r, c \leq 10$) 的网格, 黑格用 “*” 表示, 每个白格都填有一个字母。如果一个白格的左边相邻位置或者上边相邻位置没有白格 (可能是黑格, 也可能出了网格边界), 则称这个白格是一个起始格。

首先把所有起始格按照从上到下、从左到右的顺序编号为 1, 2, 3, ..., 如图 3-7 所示。

1	2	3		4	5	6
	7			8		
9			10		11	
12			13	14		
	15	16		17		
18				19	20	

图 3-7 r 行 c 列网格

接下来要找出所有横向单词 (Across)。这些单词必须从一个起始格开始, 向右延伸到一个黑格的左边或者整个网格的最右列。最后找出所有竖向单词 (Down)。这些单词必须从一个起始格开始, 向下延伸到一个黑格的上边或者整个网格的最下行。输入输出格式和样例请参考原题。

习题 3-7 DNA 序列 (DNA Consensus String, ACM/ICPC Seoul 2006, UVa1368)

输入 m 个长度均为 n 的 DNA 序列, 求一个 DNA 序列, 到所有序列的总 Hamming 距离尽量小。两个等长字符串的 Hamming 距离等于字符不同的位置个数, 例如, ACGT 和 GCGA 的 Hamming 距离为 2 (左数第 1, 4 个字符不同)。

输入整数 m 和 n ($4 \leq m \leq 50, 4 \leq n \leq 1000$), 以及 m 个长度为 n 的 DNA 序列 (只包含字母 A, C, G, T), 输出到 m 个序列的 Hamming 距离和最小的 DNA 序列和对应的距离。如有多解, 要求为字典序最小的解。例如, 对于下面 5 个 DNA 序列, 最优解为 TAAGATAC。

TATGATAC
TAAGCTAC
AAAGATCC
TGAGATAC
TAAGATGT

习题 3-8 循环小数 (Repeating Decimals, ACM/ICPC World Finals 1990, UVa202)

输入整数 a 和 b ($0 \leq a \leq 3000$, $1 \leq b \leq 3000$), 输出 a/b 的循环小数表示以及循环节长度。例如 $a=5$, $b=43$, 小数表示为 $0.(116279069767441860465)$, 循环节长度为 21。

习题 3-9 子序列 (All in All, UVa 10340)

输入两个字符串 s 和 t , 判断是否可以从 t 中删除 0 个或多个字符 (其他字符顺序不变), 得到字符串 s 。例如, $abcde$ 可以得到 bce , 但无法得到 dc 。

习题 3-10 盒子 (Box, ACM/ICPC NEERC 2004, UVa1587)

给定 6 个矩形的长和宽 w_i 和 h_i ($1 \leq w_i, h_i \leq 1000$), 判断它们能否构成长方体的 6 个面。

习题 3-11 换低挡装置 (Kickdown, ACM/ICPC NEERC 2006, UVa1588)

给出两个长度分别为 n_1, n_2 ($n_1, n_2 \leq 100$) 且每列高度只为 1 或 2 的长条。需要将它们放入一个高度为 3 的容器 (如图 3-8 所示), 问能够容纳它们的最短容器长度。



图 3-8 高度为 3 的容器

习题 3-12 浮点数 (Floating-Point Numbers, UVa11809)

计算机常用阶码-尾数的方法保存浮点数。如图 3-9 所示, 如果阶码有 6 位, 尾数有 8 位, 可以表达的最大浮点数为 $0.11111111_2 \times 2^{111111_2}$ 。注意小数点后第一位必须为 1, 所以一共有 9 位小数。

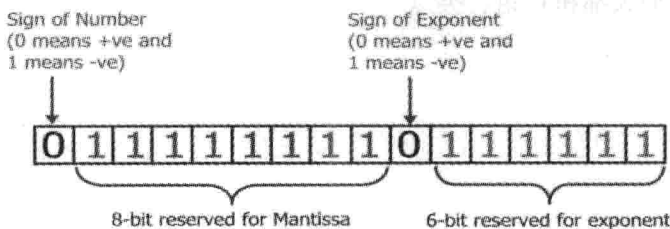


图 3-9 阶码-尾数保存浮点数

这个数换算成十进制之后就是 $0.998046875 \times 2^{63} = 9.205357638345294 \times 10^{18}$ 。你的任务是根据这个最大浮点数, 求出阶码的位数 E 和尾数的位数 M 。输入格式为 AeB , 表示最大浮点数为 $A \times 10^B$ 。 $0 < A < 10$, 并且恰好包含 15 位有效数字。输入结束标志为 $0e0$ 。对于每组数据, 输出 M 和 E 。输入保证有唯一解, 且 $0 \leq M \leq 9$, $1 \leq E \leq 30$ 。在本题中, $M+E+2$ 不必为 8 的整数倍。

3.4.5 小结

本节介绍的语法和库函数都是很直观的, 但是书中的程序理解起来比第 2 章复杂了很多, 原因在于变量突然多了很多。每当用到 $a[i]$ 或者 $s[i]$ 这样的元素时, 应该问自己: “ i 等于多少? 它有什么实际含义吗?” 作为数组下标, i 经常代表 “当前考虑的位置”, 或者与

另一个下标 j 一起表示“当前考虑的子串的起点和终点”。

数组和字符串往往意味着大数据量，而处理大数据量时经常会遇到“访问非法内存”的错误。在语法上，C 语言并不禁止程序访问非法内存，但后果难料。这在理论上可以通过在访问数组前检查下标是否合法来缓解，但程序会比较累赘；另一种技巧是适当把数组空间定义得较大，特别是不清楚数组应该开多大时。只要内存够用，开大一点没关系。顺便说一句，数组的大小可以用 `sizeof` 在编译时获得（它不是一个函数），它经常被用在 `memset`、`memcpy` 等函数中。有的函数并没有做大小检查，因而存在缓冲区溢出漏洞。本章中只讲了 `gets`，但其实 `strcpy` 也有类似问题——如果源字符串并不是以“\0”结尾的，复制工作将可能覆盖到缓冲区之外的内存。这也提醒我们：如果按照自己的方式处理字符串，千万要保证它以“\0”结尾。

在数组和字符串处理程序中，下标的计算是极为重要的。为了方便，很多人喜欢用“++”等可以修改变量（有副作用）的运算符，但千万注意保持程序的可读性。一个保守的做法是如果使用这种运算符，被影响的变量在整个表达式中最多出现一次（例如，`i = i++` 就是不允许的）。

理解字符编码对于正确地使用字符串是至关重要的。算法竞赛中涉及的字符一般是 ASCII 表中的可打印字符。对于中文的 GBK 编码，简单的实验将得出这样的结论：如果 `char` 值为正，则是西文字符；如果为负，则是汉字的前一半（这时需要再读一个 `char`）。这个结论并不是普遍成立的（在某些环境下，`char` 类型是非负的），但在大多数情况下，这样做是可行的。关于字符，另一个有意思的知识是转义序列——几乎所有编程语言都定义了自己的转义序列，但大都和 C 语言类似。